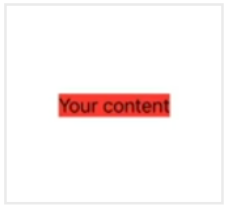
Swift and Swift UI Learning

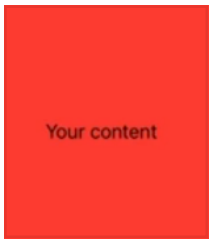
Swift UI

2 Flags project
Spacer()

VStack (alignment: .leading, spacing: 20)
Also there's HStack and ZStack (overlapping stuff)
	Logic for ZStack is bottom elements get drawn on top (last)

Colors
.background(.red) vs Color.red





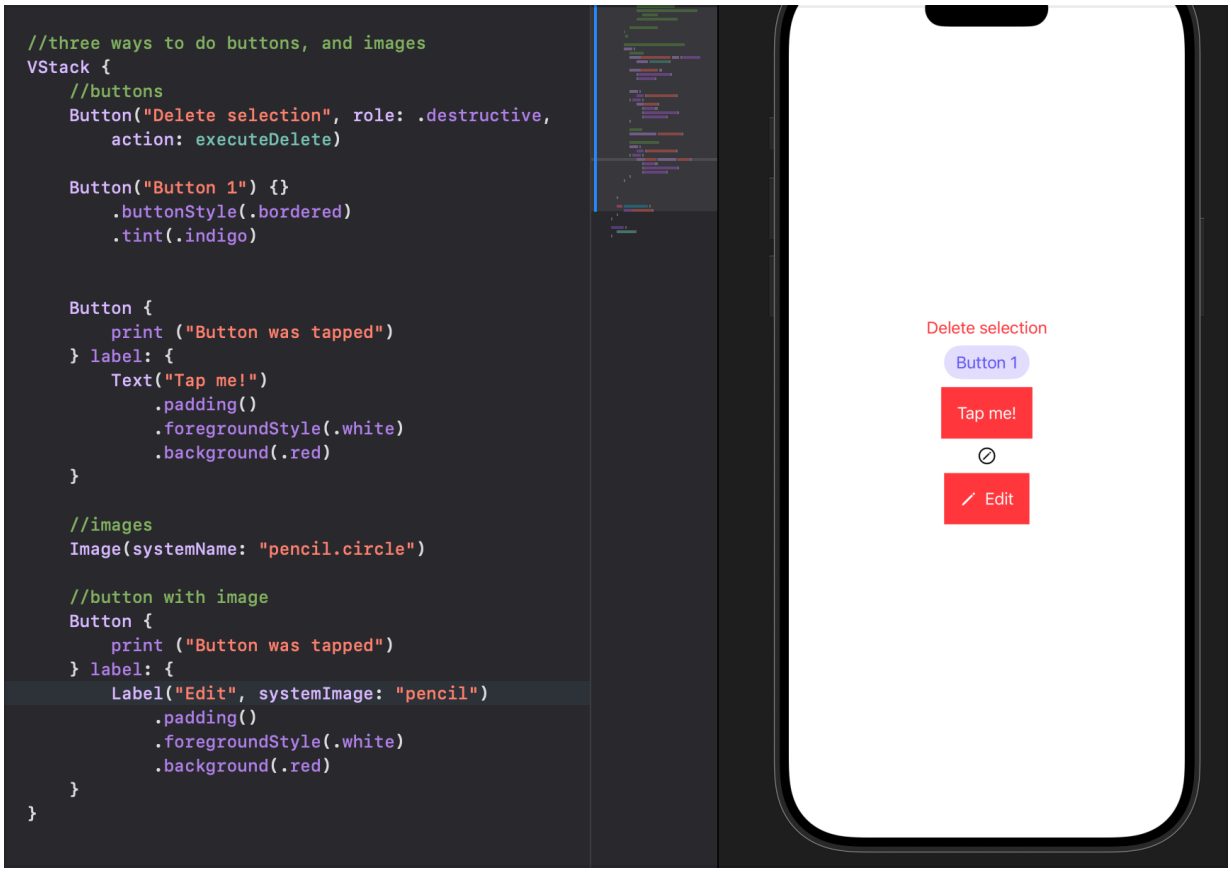
Can also use Color.primary for black, shifts based on light/dark mode, or Color.secondary for gray
	Can also do Color(red: 0, green: 0, blue: 0)

.frame to specify dimensions
.ignoresSafeArea		Ignores safe area of screen to fill whole bg

Gradients, 4 types

```
ZStack {

    LinearGradient(stops: [Gradient.Stop(color: .white, location:
        0.45), Gradient.Stop(color: .black, location: 0.55)],
        startPoint: .top, endPoint: .bottom)

    RadialGradient(colors: [.blue, .black], center: .center,
        startRadius: 20, endRadius: 200)

    AngularGradient(colors: [.red, .yellow, .green, .blue, .purple,
        .red], center: .center)


    Text("Hi")
        .foregroundStyle(.white)
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(.red.gradient)

    .ignoresSafeArea()
}
```

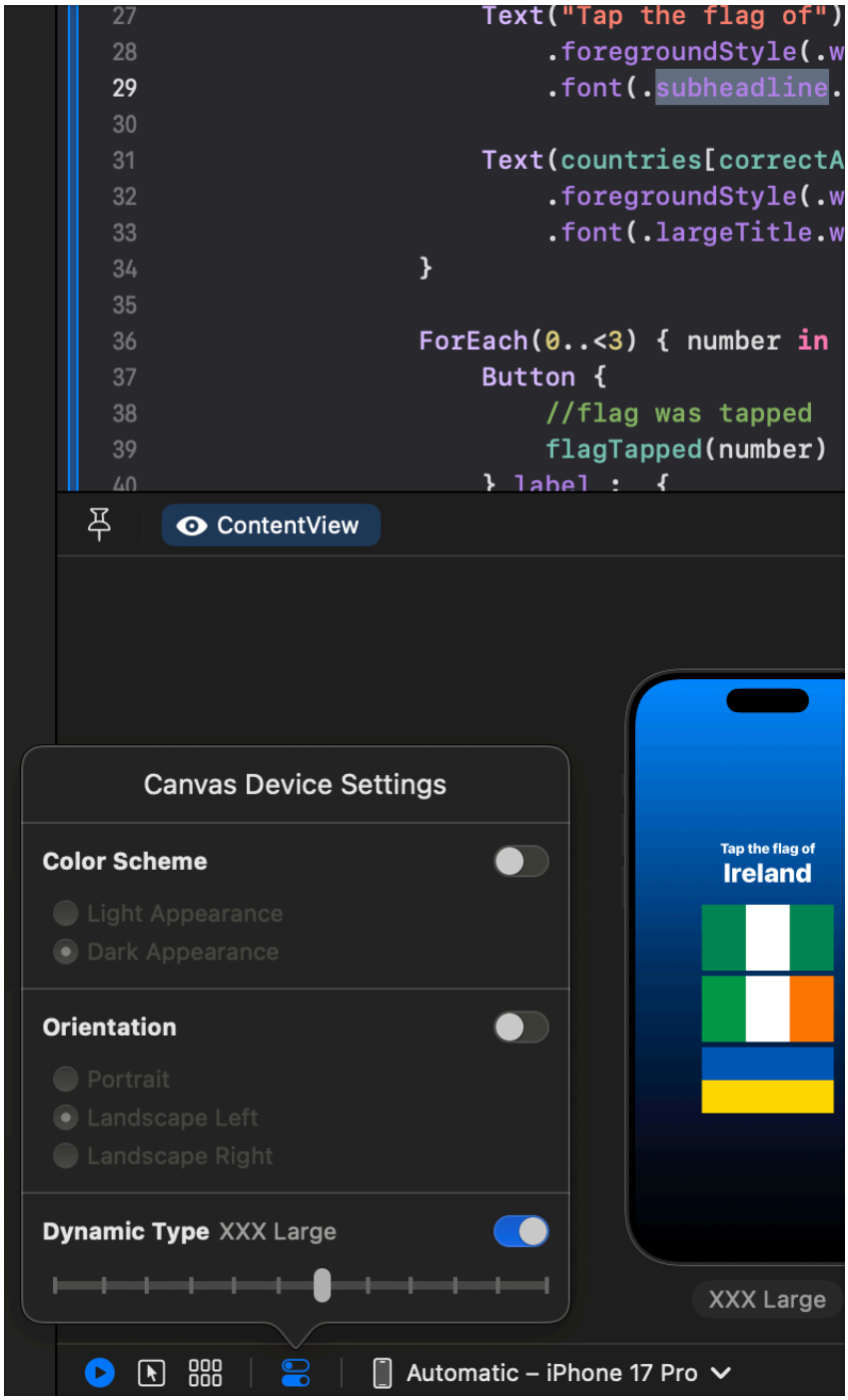
Buttons and images



Alerts

```
Button("Show alert") {
    showingAlert = true
}
    .alert("Important message", isPresented:
        $showingAlert) {
        Button("OK") { }
        Button("OK", role: .cancel) { }
    } message: {
        Text("Please read this")
    }
```

Changing default for encoded sizes like “subheadline”



1 WeSplit project

Basics swift structure

Files

WeSplitApp.swift	launches app
ContentView.swift	UI, this is what we edit
Assets.xcassets	assets catalog used directly in app
Preview Content	another assets catalog, example images used to see what app will look like

Code in ContentView.swift

```
import SwiftUI

struct ContentView: View {
   var body: some View {
      VStack {
         Image(systemName: "globe")
            .imageScale(.large)
            .foregroundStyle(.tint)
         Text("Hello, world!")
      }
      .padding()
   }
}

#Preview {                          //used purely to preview app in canvas
   ContentView()
}
```

Navigation bar, forms and sections

```
struct ContentView: View {
    var body: some View {
        NavigationView {
            Form {
                Section {
                    Text("Hello, world!")
                    Text("Hello, world!")
                }
                Section {
                    Text("Hello, world!")
                }
            }
            .navigationTitle("This is a title")
            .navigationBarTitleDisplayMode(.inline)
        }
    }
}
```

@State allows you to work around limitations of structs - not being able to modify variables

```
@State var tapCount = 0
```

Two-way binding Telling Swift how to both read and write a value
Using $ to mark variable as mutable, indicates read AND able to be rewritten

Forms and for each
    id: \.self uses the item selected in the array “Hermoine”, “Harry”, “Ron” as its own id, since they are all unique

```
var body: some View {
    NavigationStack {
        Form {
            Picker("Select your student", selection: $selectedStudent) {
                ForEach(students, id: \.self) {
                    Text($0)
                }
            }
        }
        .navigationTitle("Select a Student")
    }
}
```

Swift

LinkedIn Learning

7 Enums, protocols, errors
Enumerations Allow you to put related values in a group
    Go well together with switch statements
Syntax

```
// Declaring an enum
enum GameState {
    case Completed
    case Initializing
    case LoadingData
}

//enum GameState { case Completed, Initializing, LoadingData }
```

Using enums with switch statements

```
// Storing and switching on an enum value
var currentState = GameState.Completed
print("Current state is \(currentState)")

switch currentState {
case .Completed:
    print("Completed processing all tasks...")
case .Initializing:
    print("Still initializing data...")
case .LoadingData:
    print("Player data correctly unpacked...")
@unknown default:
    print("Unknown game state detected...")
}
```

@unknown is just a more explicit way of declaring default state, so it's safer

Can also define enum cases with raw values like so

```
// Raw values
enum NonPlayableCharacters: String {
    case Villager = "Common, not much useful info there"
    case Blacksmith = "One per village, will have quest information"
    case Merchant = "No limit per village, will make you cool stuff"
}
```

Or associated values like so

```
// Associated values
enum PlayerState {
    case Alive
    case KO(level: Int)
    case Unknown(debugError: String)
}
```

```
enum PlayerState {
    case Alive
    case KO(level: Int)
    case Unknown(debugError: String)

    func evaluateCase() {
        switch self {
        case .Alive:
            print("Still kicking!")
        case .KO(let restartLevel):
            print("Sorry, back to level \(restartLevel) for you...")
        case .Unknown(let message):
            print(message)
        default:
            print("Unknown state encountered...")
        }
    }
}

PlayerState.KO(level: 1).evaluateCase()
```

Protocols
Like a template of methods, properties, and other requirements that can be adopted by classes or structs

You'll get to autofill the "template" when you declare a class according to that protocol
Syntax

```
// Declare a protocol
protocol Collectable {
    var name: String { get }
    var price: Int { get set }

    init(withName: String, startingPrice: Int)
    func collect() -> Bool
}
```

You can extend a protocol, so you don't have to conform completely to it

Extensions
Extend existing class, struct, enum or protocol

Errors

Error protocol doesn't have anything in it, it's just a flag to mark something as checking for errors
throws keyword make a function as throwable (can throw an error)

```
// Error enum
enum DataError: Error {
    case EmptyPath
    case InvalidPath
}

// Throwing functions
func loadData(path: String) throws {
    guard path.contains("/") else {
        throw DataError.InvalidPath
    }

    guard !path.isEmpty else {
        throw DataError.EmptyPath
    }
}
```

Handle errors with try keyword in do-catch statements:

```
// Do-Catch statements
do {
    try loadData(path: playerDataPath)
    print("Data fetch successful!")
} catch is DataError {
    print("Invalid or empty path detected...")
} catch {
    print("Unknown error detected...")
}
```

Propagating errors/passing errors to other functions:

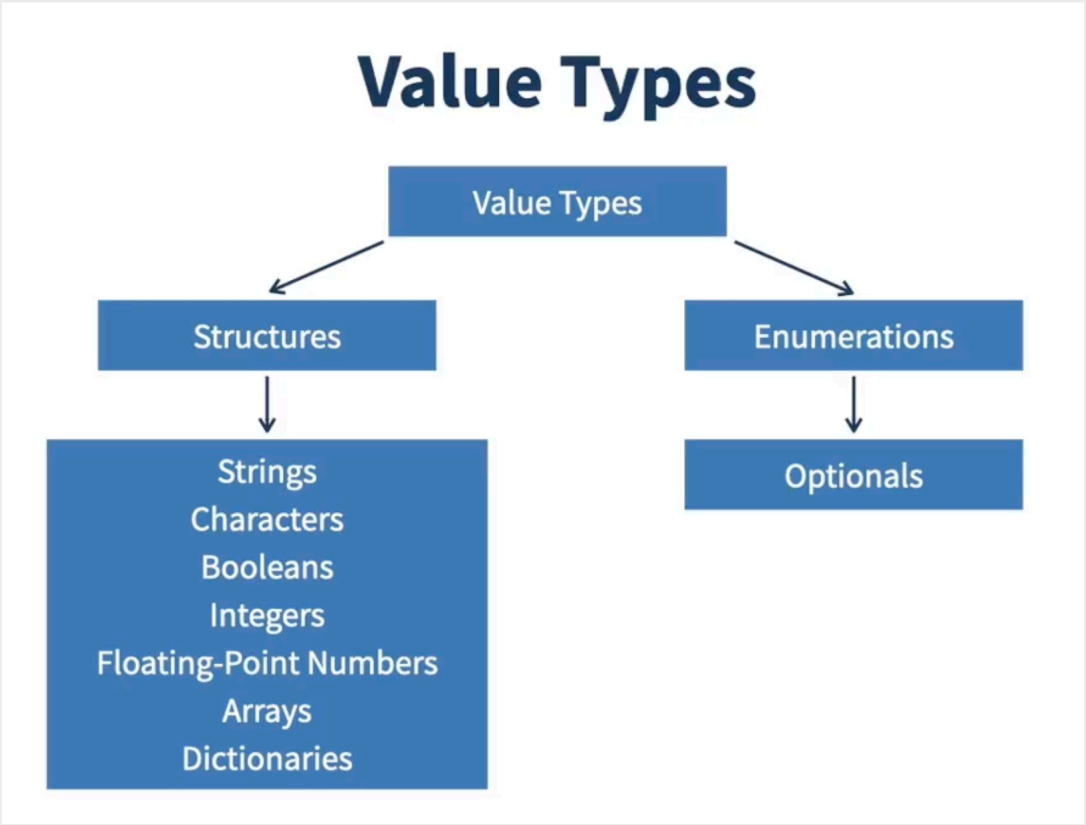
```swift
// Propagating errors
func propagateDataError() throws {
    try loadData(path: playerDataPath)
}

do {
    try propagateDataError()
    print("Propagated data fetch successful!")
} catch DataError.EmptyPath {
    print("Empty path detected...")
} catch DataError.InvalidPath {
    print("Invalid path detected...")
} catch {
    print("Unknown error...")
}
```

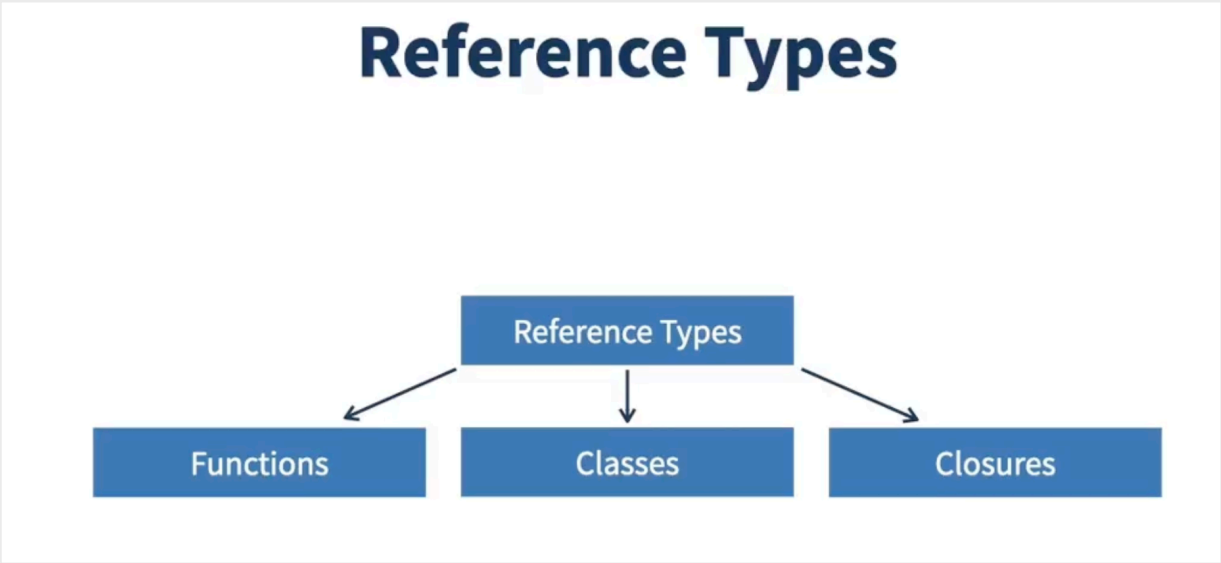
6 Classes, structs

Value types vs reference types

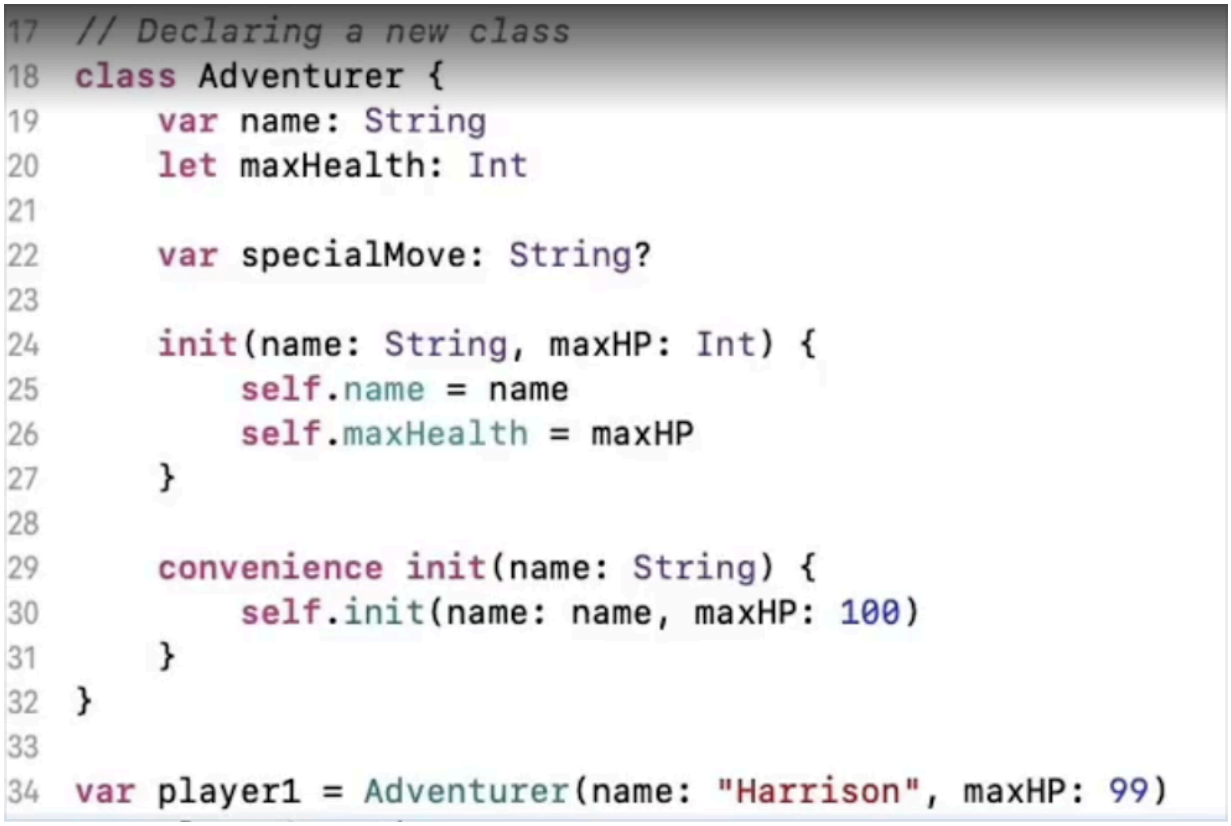
- Depends on how they're passed / assigned
- Value types passed by copy
- Reference types passes by reference, so directly. If one changes, all change

Structs are like a simpler class





Class syntax



```swift
// Declaring a new class
class Adventurer {
    var name: String
    let maxHealth: Int

    var specialMove: String?

    init(name: String, maxHP: Int) {
        self.name = name
        self.maxHealth = maxHP
    }

    convenience init(name: String) {
        self.init(name: name, maxHP: 100)
    }
}

var player1 = Adventurer(name: "Harrison", maxHP: 99)
```

init vs convenient init

convenient init just makes it more convenient lol

Standard `init`:

> Designated initializers are the primary initializers for a class. A designated initializer fully initializes all properties introduced by that class and calls an appropriate superclass initializer to continue the initialization process up the superclass chain.

`convenience init`:

> Convenience initializers are secondary, supporting initializers for a class. You can define a convenience initializer to call a designated initializer from the same class as the convenience initializer with some of the designated initializer's parameters set to default values. You can also define a convenience initializer to create an instance of that class for a specific use case or input value type.

per the Swift Documentation

Make get and set events if you want

Subclass syntax

```
// Subclass
class Ranger: Adventurer {
    var classAdvantage: String

    override class var credo: String {
        return "To the King!"
    }

    init(name: String, advantage: String) {
        self.classAdvantage = advantage
        super.init(name: name, maxHP: 150)
    }
}
```

Struct syntax

```
// Declaring a new struct
struct Level {
    let levelID: Int
    var levelObjectives: [String]

    var levelDescription: String {
        return "Level ID: \(self.levelID)"
    }

    init(id: Int, objectives: [String]) {
        self.levelID = id
        self.levelObjectives = objectives
    }
}
```

Chaining optionals together

```
if let ownerInfo = rareDagger.previousOwner?.returnOwnerInfo() {
    print("Owner found!")
} else {
    print("No owner in our records...")
}

if let gemstoneObjective = questDirectory["Fetch Gemstones"]?["Objective"] {
    print(gemstoneObjective)
}
```

5 Functions
Basic syntax

```
// Basic function
func findNearestAlly(level: Int) -> String {
    print("Searching for ally above level \(level)")
    return "Argus"
}

var ally = findNearestAlly(level: 5)
```

Function overloading Multiple functions with the same name as long as the function type (signature) differs

```
// Overloaded functions
func attackEnemy(damage: Int) {
    print("Attacking for \(damage)")
}

func attackEnemy(damage: Double, weapon: String) -> Bool {
    let attackSuccess = true
    print("Attacking enemy for \(damage) with \(weapon)")

    return attackSuccess
}

attackEnemy()
attackEnemy(damage: 34)
```

Using optionals in functions

```swift
// Optional return value
func setupArenaMatch() -> Bool? {
    return nil
}

if let initSuccess = setupArenaMatch() {
    print("Level initialized: \(initSuccess)")
} else {
    print("Something went terribly wrong...")
}
```

Functions as parameters

```swift
// Functions as parameters
func dealDamage(baseDamage: Int, bonusDamage: (Int) -> Int) {
    let bonus = bonusDamage(baseDamage)
    print("Base Damage: \(baseDamage)\nBonus Damage: \(bonus)")
}

dealDamage(baseDamage: 55, bonusDamage: computeBonusDamage)
```

Closures syntax (async functions

```swift
// Defining closures
var closure: () -> () = {}

// Initializing closures
var computeBonusDamage = { (base: Int) -> Int in
    return base * 4
}

computeBonusDamage(25)
```

Middle part syntax is same as here

```
// Initializing closures
var computeBonusDamage = { base in
    return base * 4
}
```

Type alias
It's just a way to name a series of parameters so you don't have to write them out over and over again in functions. That's one use case at least

```
// Type alias as a return value
typealias AttackTuple = (name: String, damage: Int, rechargeable: Bool)

var sunStrike: AttackTuple = ("Sun Strike", 45, false)

func levelUpAttack(baseAttack: AttackTuple) -> AttackTuple {
    let increasedAttack: AttackTuple = (baseAttack.name, baseAttack.damage + 10, true)
    return increasedAttack
}
```

4 Control flow (for loops etc)

Three dots for closed ranges

if statements
    if statements don't require parentheses
    Use "if let item = itemGathered" instead of "if item == itemGathered"

Using optional binding to unwrap optionals in if statements

```
// 2
if let leftWeapon = lefthandWeapon, let rightWeapon = righthandWeapon {
    print("Looks like your \(leftWeapon) and \(rightWeapon) are an even match for me!")
} else {
    print("I didn't bring enough hardware for this...")
}
```

for in loops
    for stringCharacter in playerGreeting
    for weaponValues in weapons.values
    for (weapon, damage) in weapons
    for indexNumber in 1...10 {
    for armor in armorTypes[0...] {

Arrays
    Dictionaries values
Dictionaries keys and values
    For using indices with closed ranges
    For using indices with half open range

for indexNumber in 1..<10 { For using indices up to a certain value
for armor in armorTypes[..<2] { For using indices up to a certain value with inferred first value (0 in this case)

while loops
while playerHealth > 0 {
Have to manually decrease count with playerHealth -= 1

repeat-while loops
Same as while loops except expression is checked at end of statement instead of beginning
Useful for when you know the first iteration will pass the while condition

repeat {
playerHealth -= 1
print("HP at \(playerHealth)")
} while playerHealth > 0

switch statement
switch initial {
case "H":
print ("harold?")
case "A":
print ("audrey?")
default:
print ("idk")
}

You can insert ranges too, like case (1...14, 20..<25)

More complex switch variations

```
// Complex variations
switch (mp, hp) {
case (15, 10):
    print("Great job")
case (1...15, 20..<25):
    print("Ranges are the best!")
case (let localMP, let localHP) where localMP + localHP > 20:
    print(localMP, localHP)
default:
    print("I've got nothing...")
}
```

Value binding with let here

```
case (let localExp) where localExp > 500:
    print("Time to level up!")
```

Guard statement

It’s like a stricter if statement, that’s more concise

```
// Test variables
let shopItems = ["Magic wand": 10, "Iron Helm": 5, "Excalibur": 1000]
let currentGold = 16

// Guard statement with for-in loop
for (item, price) in shopItems {
    guard currentGold >= price else {
        print("You can't afford the \(item)")
        continue
    }

    print("Go ahead, the \(item) is yours for \(price) gold!")
}
```

3 Collections

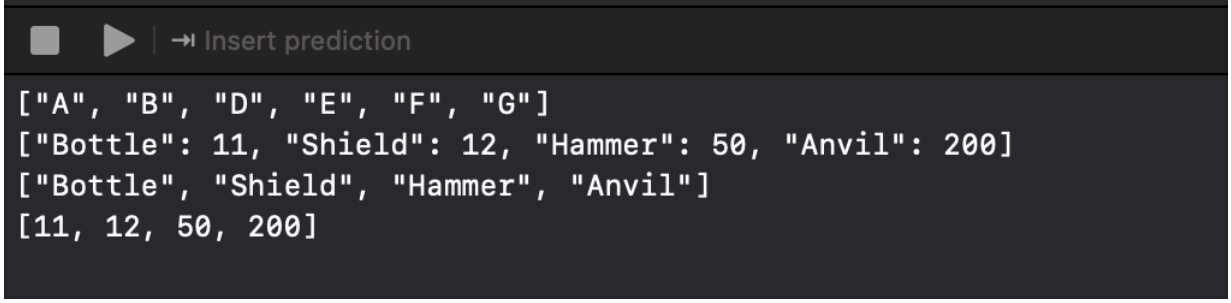
Arrays

```swift

// Creating arrays
var levelDifficulty: [String] = ["Easy", "Medium", "Hard"]

var emptyArray: [String] = []
var emptyArray2: Array<String> = Array<String>()
    //both above empty array

// Count and isEmpty
levelDifficulty.count
levelDifficulty.isEmpty

// Accessing array values
var mostDifficult = levelDifficulty[2]
levelDifficulty[2] = "Utter Ridiculousness"
```

```swift
// Changing & appending items
var characterClasses = ["A", "B", "C"]
characterClasses.append("D")
characterClasses += ["E", "F"]

//Inserting and removing items
characterClasses.insert("G", at: 1)
characterClasses.remove(at: 3)

// Ordering and querying values
characterClasses.reverse()
var reversedClasses = characterClasses.reversed()

characterClasses.sort()
var sortedClasses = characterClasses.sorted()

print(characterClasses)

// 2D arrays and subscripts
var skillTree: [[String]] = [
    ["A", "B", "C"],
    ["D", "E", "F"]
]

var attackTreeSkill = skillTree[0][2]
```

Dictionaries

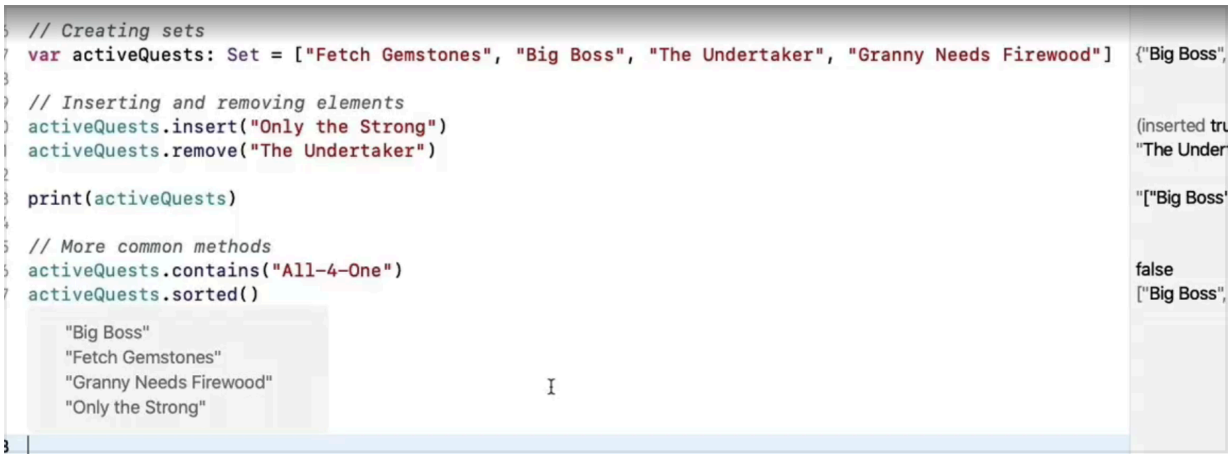
```swift
// Creating dictionaries
var blacksmithShop: [String: Int] = ["Bottle": 10, "Hammer":
    50, "Anvil": 200]

// Accessing and modifying values
var shieldPrice = blacksmithShop["Shield"]
blacksmithShop["Bottle"] = 11
blacksmithShop["Shield"] = 12

print(blacksmithShop)

// All keys and values
let allKeys = [String](blacksmithShop.keys)
let allValues = [Int](blacksmithShop.values)
print(allKeys)
print(allValues)
```




```
["A", "B", "D", "E", "F", "G"]
["Bottle": 11, "Shield": 12, "Hammer": 50, "Anvil": 200]
["Bottle", "Shield", "Hammer", "Anvil"]
[11, 12, 50, 200]
```

```swift
// Caching and overwriting items
var playerStats: [String: Int] = ["HP": 100, "Attack": 35]
var oldValue = playerStats.updateValue(30, forKey: "Attack")

playerStats = ["Evasion": 12, "MP": 55] //overrides dictionary

// Caching and removing items
var removedValue = playerStats.removeValue(forKey: "HP")
//playerStats["Gold"] = nil

// Nested dictionaries
var questBoard = ["Quest 1": ["Reward": "Item X", "Difficulty": 1],
                  "Quest 2": ["Reward": "Item Y", "Difficulty": 2]]

var gemstoneObjective = questBoard["Fetch gemstones"]?["Reward"]
//? breaks the chain if the compiler knows the second part doesnt exist
```

Sets
Good for data that doesn't need to be ordered




```swift
// Test variables
var activeQuests: Set = ["Fetch Gemstones", "Big Boss", "The Undertaker", "Granny Needs Firewood"]
var completedQuests: Set = ["Big Boss", "All-4-One", "The Hereafter"]

// Set operations
var commonQuests = activeQuests.intersection(completedQuests)
var differentQuests = activeQuests.symmetricDifference(completedQuests)
var allQuests = activeQuests.union(completedQuests)
var clippedQuests = activeQuests.subtracting(completedQuests)
```

Symmetric difference is everything outside the intersection

Tuples
Takes a group of values and stores them as a single value
Good for returning values

Not good for complex data structures

```
// Simple tuple
var uppercutAttack: (String, Int, Bool) = ("Uppercut Smash", 25, true)
uppercutAttack.0
uppercutAttack.1
uppercutAttack.2

var (attack, damage, rechargeable) = uppercutAttack
attack
damage
rechargeable

// Naming tuple values
var planetSmash = (name: "Planet Smash", damage: 45, rechargeable: true)
planetSmash.rechargeable

// Naming values with type annotation
var shieldStomp: (name: String, damage: Int, rechargeable: Bool)
shieldStomp.damage = 100
```

2 Variables
To cross-check against LinkedIn Learning

- Day 1 – Variables, constants, strings, and numbers
- Day 2 – Booleans, string interpolation, and checkpoint 1
- Day 3 – Arrays, dictionaries, sets, and enums
- Day 4 – Type annotations and checkpoint 2
- Day 5 – If, switch, and the ternary operator
- Day 6 – Loops, summary, and checkpoint 3
- Day 7 – Functions, parameters, and return values
- Day 8 – Default values, throwing functions, and checkpoint 4
- Day 9 – Closures, passing functions into functions, and checkpoint 5
- Day 10 – Structs, computed properties, and property observers
- Day 11 – Access control, static properties and methods, and checkpoint 6
- Day 12 – Classes, inheritance, and checkpoint 7
- Day 13 – Protocols, extensions, and checkpoint 8
- Day 14 – Optionals, nil coalescing, and checkpoint 9

type safe language
    compiler infers variable type

Log stuff with print()
Combining initializing and declaring

```
// Declaring strings
var activeString: String = "Retrieving the cart!"
```

```swift

// String interpolation
var color: String = "blue"
var colorSentence: String = "My favorite color is \(color)!"

```

Strings
String.count
String.isEmpty
String.contains(“a”)
String.appent(contentsOf: “Hello!”)
String.removeLast() String.removeFirst() String.removeAll()
String.split(separator: “,”)

Converting types
Explicit and inferred conversions

Declaring booleans

```swift
// Test variable
var isActive: Bool = false
var canMove = false
```

Optionals variable types that can store a value or nil (null) value
Forced unwrapping converting an optional into a normal variable

Cmd+RMB Opens action menu

1
Basics
- Compiled
- Developed to work with Objective-C modules and frameworks
- Project can have both Object-C and Swift side by side